A

AuraOS

Diseño Arquitectónico y Conceptualización de un Sistema Operativo

Arquitectura · UX/UI · Análisis de Valor

Agenda

01 El problema de los SO actuales

02 Identidad y filosofía de AuraOS

03 Arquitectura de software

04 Seguridad y sistema de archivos

05 Gestión de hardware

06 Análisis de valor

07 Requerimientos de hardware

08 Interfaz ZenUI

09 Conclusiones y fuentes

Limitaciones de los SO Monolíticos

Superficie de ataque masiva

Los núcleos monolíticos superan las decenas de millones de líneas de código, imposibles de auditar completamente.

Fallo único, colapso total

Un driver defectuoso ejecutándose en kernel space puede derribar todo el sistema operativo.

Seguridad basada en identidad

Las ACL tradicionales vinculan permisos al usuario, no al proceso — un proceso comprometido hereda todos los privilegios.

Complejidad heredada

Décadas de retrocompatibilidad acumulan capas de abstracción que penalizan el rendimiento y la mantenibilidad.

Identidad y Filosofía

Nombre

Aura Operating System (AuraOS) — evoca una capa protectora, invisible y omnipresente que envuelve al sistema.

Misión

Disrupción arquitectónica para entornos de misión crítica: servidores financieros, infraestructura IoT, sistemas de defensa.

Logo

Cinta de Möbius estilizada — seguridad continua, sin inicio ni fin. Paleta en tonos índigo profundo con acentos ámbar.



SECCIÓN I

Arquitectura de Software

Micronúcleo de Tercera Generación



~10,000 líneas de código

Frente a los 30M+ de un kernel monolítico — auditable y verificable formalmente.

Inspirado en seL4

El único kernel con prueba matemática de corrección funcional completa.

Mínimo privilegio

El kernel solo hace tres cosas. Todo lo demás corre aislado en procesos de usuario.

Verificación Formal

Prueba matemática asistida por computadora que demuestra que la implementación en C cumple su especificación abstracta — sin ambigüedad.

1. Especificación formal del comportamiento esperado
2. Prueba de que el código implementa esa especificación
3. Eliminación de clases enteras de vulnerabilidades

0

bugs por diseño

Buffer overflows, null dereferences y deadlocks son matemáticamente imposibles en el kernel verificado.

IPC de Alto Rendimiento

Los micronúcleos de primera generación (Mach) sufrían penalización severa por la sobrecarga del IPC. AuraOS adopta las optimizaciones de seL4:

Registro directo

Mensajes cortos se transfieren directamente en registros de CPU, evitando copias a memoria.

Fast-path

Ruta de ejecución optimizada en ensamblador para las operaciones IPC más frecuentes.

Memoria compartida

Para volúmenes grandes de datos, regiones de memoria compartida evitan la copia por completo.

Seguridad Basada en Capacidades

ACL Tradicional

Permisos vinculados a la identidad del usuario. Un proceso comprometido hereda todos los privilegios del usuario.

Capacidades (AuraOS)

Tokens criptográficos infalsificables otorgados por proceso. Acceso solo a los recursos explícitamente autorizados.

FLUJO DE AUTORIZACIÓN



Sistema de Archivos AuraFS

Servidor en espacio de usuario

Opera como proceso aislado, comunicándose con el kernel exclusivamente por IPC. Un fallo en AuraFS no colapsa el sistema.

Indexación con B-trees

Estructura jerárquica autobalanceada que garantiza O(log n) en búsquedas, inserciones y eliminaciones.

Copy-on-Write (CoW)

Los datos nunca se sobreescriben en su ubicación original. Cada escritura crea una nueva versión, permitiendo rollback atómico.

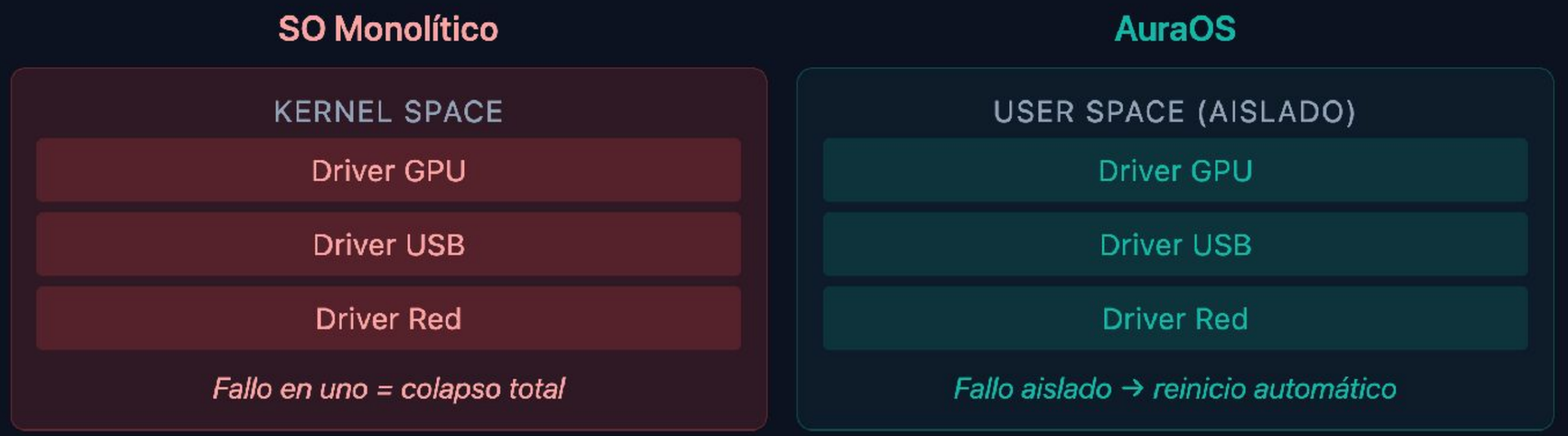
Snapshots instantáneos

Gracias al CoW, crear un snapshot del estado completo del filesystem es una operación de costo O(1).

SECCIÓN II

Gestión de Hardware

Controladores en Espacio de Usuario



Contención de fallos

Excepciones, desbordamientos y punteros nulos quedan contenidos en el espacio de memoria del driver afectado.

IOMMU obligatorio

El hardware restringe el acceso DMA de cada dispositivo, previniendo escrituras fuera de la región asignada.

Gestión de Memoria

Memoria virtual multinivel

Explotación completa de la MMU del procesador con tablas de páginas jerárquicas para aislamiento total entre procesos.

Paginadores en espacio de usuario

Resuelven page faults, aplican algoritmos de reemplazo (LRU, Clock) y gestionan swap — todo fuera del kernel.

Aislamiento criptográfico

Las regiones de memoria están protegidas por las mismas capacidades que gobiernan el acceso a recursos del sistema.

Coherencia de caché

Diseño consciente de las organizaciones multinúcleo para mantener consistencia en memorias caché L1/L2/L3.

Servidores de Personalidad

La flexibilidad del micronúcleo permite ejecutar aplicaciones diseñadas para otros SO mediante servidores de personalidad en espacio de usuario.



Análisis de Valor

Ventajas

Seguridad determinística — aislamiento formal entre todos los componentes

Tolerancia a fallos — drivers reiniciables sin afectar el sistema

Rendimiento optimizado — sin peso de subsistemas heredados innecesarios

Interfaz transparente — complejidad oculta tras UX pulida (ZenUI)

Desafíos

Sobrecarga de IPC — latencia adicional frente a llamadas directas en monolíticos

Ecosistema de apps — dependencia de servidores de personalidad para compatibilidad

Curva de aprendizaje — paradigma de capacidades requiere repensar el desarrollo

Efecto red — masa crítica de usuarios y desarrolladores como barrera histórica

Requerimientos de Hardware

Componente	Mínimo	Recomendado
CPU	64-bit, 2 núcleos, 1.5 GHz, MMU	4+ núcleos, 2.5 GHz, VT-x/VT-d
RAM	4 GB	16 GB+ DDR4/DDR5
Almacenamiento	64 GB SSD	256 GB+ NVMe
GPU	Integrada, 2D básica, 1280×720	Dedicada, Vulkan, 1920×1080+
Seguridad HW	TPM 1.2, IOMMU básico	TPM 2.0, IOMMU robusto, Secure Boot

Interfaz Gráfica ZenUI

Compositor Wayland nativo

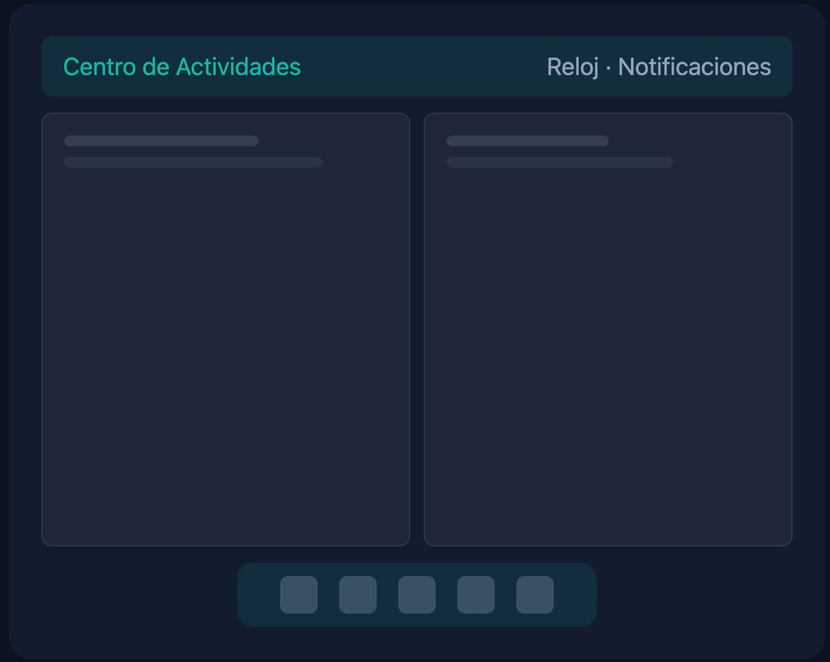
Reemplaza X11 por completo. Cada app renderiza en su propio buffer aislado criptográficamente.

Glassmorphism funcional

Desenfoques gaussianos que comunican profundidad y jerarquía, no decoración. Tipografía sans-serif geométrica.

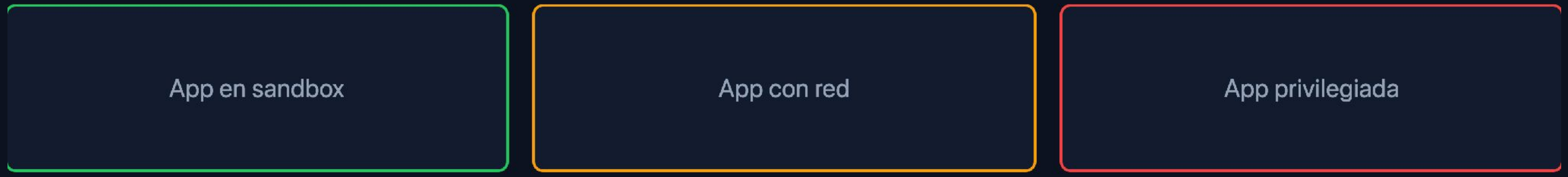
Gestor de ventanas híbrido

Apilamiento libre (floating) con soporte de tiling automático para flujos de trabajo técnicos.



Codificación de Confianza Visual

El estado de seguridad de cada aplicación es visible para el usuario mediante indicadores cromáticos en los bordes de ventana.



Confianza alta

Sin acceso a red ni filesystem externo

Confianza media

Acceso a red autorizado por capability

Confianza baja

Permisos elevados, monitoreo activo

Conclusiones

1. La verificación formal del kernel es viable y elimina clases enteras de vulnerabilidades que el testing convencional no puede detectar.
2. El aislamiento profundo de componentes (drivers, filesystem, red) aumenta la resiliencia del sistema sin sacrificar funcionalidad.
3. La brillantez arquitectónica es insuficiente sin una estrategia de ecosistema: servidores de personalidad y herramientas de desarrollo son críticos.
4. ZenUI demuestra que la seguridad criptográfica del núcleo puede coexistir con una experiencia de usuario intuitiva y visualmente coherente.

Fuentes Citadas

1. Silberschatz, Galvin, Gagne — Operating System Concepts, 10th Ed. (Wiley)

2. Tanenbaum — Modern Operating Systems

3. Klein et al. — seL4: Formal Verification of an OS Kernel (ResearchGate)

4. DTIC — Final Report AOARD 094160: Formal System Verification

5. Capability-based Addressing — Wikipedia

6. Chung — Segmentation for Capability-based Single Address Space

7. Stallings — Operating Systems: Internals and Design Principles, 9th Ed.

8. Computer Organization and Architecture — Stallings, 11th Ed.

9. Wayland Protocol Documentation (freedesktop.org)

10. Norman — The Design of Everyday Things

11. SFU Operating Systems — Textbook and References

12. Hennessy, Patterson — Computer Architecture: A Quantitative Approach